

Aegis

Runtime Credential Defense for LLM Agents

Project Plan

Gauntlet AI Capstone

June 2026

Capstone Direction: Combine ML with LLM applications
Team Assumption: Cameron Candelori, Christopher King, Jay Godfrey, Ryan Jagger
Planning Basis: Gauntlet Capstone Brief, prior Aegis proposal, AIS paper
Demo Date: June 29, 2026

Abstract

LLM agents are increasingly useful because they can act across external systems, but that usefulness requires access to credentials, tools, and untrusted content in the same operational loop. This creates a structural security risk: indirect prompt injection can steer an agent toward credential exfiltration through natural-language output, encoded transformations, low-rate multi-turn leakage, or structured tool-call arguments. Aegis is a proposed SDK-backed runtime security layer that can also run behind a lightweight gateway. It normalizes agent traffic, inspects requests, model output, and tool calls, scores exfiltration risk, and enforces configurable policy with auditable evidence. The project is grounded in the AIS research prototype, which combines activation-based credential access detection, calibrated honeytokens, and cumulative leakage accounting. Aegis adapts that research direction into a viable product architecture and focuses on a deployment gap identified by the research: structured tool-call arguments. The two-week implementation plan prioritizes a Python SDK with explicit guard methods, a FastAPI gateway wrapper, provider adapters, modular detectors, YAML policy modes, a local credential broker, canary detection, cumulative leakage scoring, Braintrust-backed evaluation with local JSONL fallback, and a small dashboard/reporting surface.

Keywords: LLM agents, prompt injection, credential exfiltration, tool-call security, runtime gateway, honeytokens, leakage accounting, AI security.

Contents

1	Introduction	1
2	Research Grounding	1
3	Product Requirements	1
3.1	Problem Statement	1
3.2	Product Vision	2
3.3	Users and Stakeholders	2
3.4	Goals	2
3.5	Non-Goals	3
3.6	Functional Requirements	3
3.7	Non-Functional Requirements	4
3.8	Success Metrics	5
4	System Architecture	5
4.1	SDK and Gateway Architecture	5
4.2	Guard Surface	6
4.3	Runtime Contracts	6
4.4	Implementation Decisions	7

5	Project Plan	7
5.1	Team Ownership	7
5.2	Milestones	8
5.3	Day-by-Day Execution	8
6	Detector and Policy Design	9
6.1	Detector Contract	9
6.2	MVP Detector Stages	10
6.3	Initial YAML Policy Scope	10
6.4	Policy Modes	10
6.5	Credential Broker Scope	11
7	Evaluation Plan	11
7.1	Scenario Categories	11
7.2	Metrics	11
7.3	Braintrust and Artifact Plan	12
7.4	Demo Metrics Table	12
8	Testing Strategy	12
8.1	Unit Tests	12
8.2	Integration and Eval Tests	13
9	Failure Modes and Guardrails	13
10	Risk Register	13
11	Demo Narrative	14
12	Limitations	14
13	Conclusion	15
A	Architecture Diagram	16

1 Introduction

The Gauntlet capstone tasks teams with building a technically ambitious system that uses AI as a core primitive rather than as a decorative feature. Aegis targets a high-stakes failure mode in agentic systems: credential exfiltration through indirect prompt injection. The project is ambitious because the problem is not merely a classification task over suspicious strings. It emerges from the architecture of useful agents. Agents must read untrusted content, maintain task context, and call tools; credentials and sensitive handles often live adjacent to attacker-controlled text or tool outputs.

The central objective is to build a working runtime gateway that makes these flows observable and enforceable. Aegis is not a complete solution to credential exfiltration. Rather, it is a practical gateway that can meaningfully raise the visibility and difficulty of leaks, especially through structured tool-call arguments, while preserving legitimate agent behavior in scripted benign scenarios.

2 Research Grounding

The primary research reference is *Caught in the Act(ivation): Toward Pre-Output and Multi-Turn Detection of Credential Exfiltration by LLM Agents* by Chauhan and Revankar [2]. The paper presents the Agentic Immune System (AIS), a research prototype that combines three complementary defenses:

1. **CIFT**: activation-based pre-output detection for open-weight models with white-box access.
2. **DP-HONEY**: calibrated honeypot generation and detection.
3. **NIMBUS**: cumulative leakage scoring across conversation turns.

As a research prototype, AIS is not a deployment-ready product. CIFT requires activation access and therefore does not directly apply to closed cloud APIs. NIMBUS is a learned leakage signal rather than a certified information-flow bound. Most importantly, credentials routed through structured tool-call arguments are outside the prototype scope and are identified as a severe blind spot.

Aegis uses those limitations as design direction. The system preserves the AIS pattern of multi-layer monitoring while shifting the deliverable toward practical runtime enforcement. The capstone contribution is not a new proof of security; it is an implemented gateway and evaluation loop that attacks a blind spot left open by the research prototype.

3 Product Requirements

3.1 Problem Statement

LLM agents can call tools, query databases, send messages, browse documents, and act across external systems. These capabilities frequently require credentials such as API keys, OAuth tokens, database passwords, or service-specific secrets. The resulting agent context may combine trusted secrets with untrusted content, including webpages, emails, retrieved documents, tool outputs, and user-provided artifacts.

An attacker can exploit this arrangement by placing indirect prompt-injection instructions in untrusted content. The agent may then be steered toward revealing credentials directly, transforming them through encodings, leaking them

slowly across turns, or placing them into structured tool-call arguments. Text-level filters remain useful, but they do not cover the full attack surface.

3.2 Product Vision

Aegis is a drop-in security layer for LLM agents. The core product is a Python SDK that owns the security logic. A lightweight platform gateway wraps that SDK for demos and integrations, exposing policy configuration, session visibility, evaluation results, and report artifacts without making the gateway itself the only integration path.

The capstone version should demonstrate five properties:

1. Agent traffic can be intercepted and normalized through explicit SDK guard points or a provider gateway.
2. Tool-call arguments can be inspected before dispatch.
3. Canary and cumulative leakage signals can be combined with policy enforcement.
4. Each non-allow decision can be explained with structured evidence and trace metadata.
5. Baseline and protected runs can be compared through repeatable evaluation artifacts.

3.3 Users and Stakeholders

Stakeholder	Need
AI platform owner	Deploy agentic workflows while reducing the risk that secrets leak through model output or tool calls.
Security engineer	Configure policy, inspect evidence, and understand why an action was allowed, warned, sanitized, blocked, or escalated.
Agent developer	Add a defense layer without redesigning the application or rewriting tool logic.
Red teamer	Run replayable attack cases and produce artifacts showing where defenses succeeded or failed.
Capstone evaluator	See a technically ambitious system working live with clear limitations and measurable outcomes.

3.4 Goals

1. Build an SDK-backed runtime gateway for observing model requests, model responses, and selected tool-call arguments.
2. Implement an Inspect–Score–Enforce pipeline with modular detectors.
3. Treat tool-call argument scanning as a first-class defense.
4. Implement calibrated canary detection and session-level leakage accounting inspired by DP-HONEY and NIMBUS.
5. Provide a live dashboard, Braintrust experiment trail, and local fallback audit artifacts that explain decisions.

6. Evaluate benign flows and three attack classes: encoded leakage, low-rate multi-turn leakage, and tool-call argument exfiltration.
7. Deliver a June 29 demo comparing a baseline agent against an Aegis-protected agent.

3.5 Non-Goals

1. We do not claim production-grade prevention of all credential exfiltration.
2. We do not make full research-grade CIFT activation probing a required MVP dependency.
3. We do not support every agent framework, model provider, or tool schema.
4. We do not build production secret storage, rotation, tenancy, billing, access control, or compliance workflows.
5. We do not create a complex policy DSL or visual policy editor.
6. We do not rely on an LLM judge as the only detector or as the source of blocking authority.
7. We do not make PyTorch, CIFT, or open-weight model hosting a hard dependency for the MVP.
8. We do not optimize for Rust gateway performance before proving the detection pipeline.

3.6 Functional Requirements

ID	Requirement	Priority	Acceptance Criteria
FR-1	SDK guard surface	P0	The Python SDK exposes request, tool-call, and response guard methods that return structured decisions and can be invoked without running the gateway.
FR-2	Provider gateway	P0	A local service receives a provider-compatible request, normalizes it through the SDK, forwards allowed or sanitized traffic to a configured upstream or mock provider, logs the response, and returns a valid response.
FR-3	Request and response normalization	P0	The system creates a normalized event representation for messages, tool calls, tool arguments, model output, session ID, trace ID, provenance, and trust boundary.
FR-4	Detector contract	P0	Each detector returns name, score, confidence, recommended action, latency, and structured evidence.
FR-5	Tool-call argument scanning	P0	At least three high-risk tool schemas are supported: <code>send_email</code> , <code>http_request</code> , and <code>query_database</code> . Suspicious values are flagged before dispatch.
FR-6	Canary generation and detection	P0	The gateway can register honeypot tokens, expose them only in model-visible context, and detect their appearance in output or tool arguments.

ID	Requirement	Priority	Acceptance Criteria
FR-7	Session leakage accounting	P0	The gateway maintains a per-session cumulative leakage score and triggers warning, blocking, or escalation thresholds.
FR-8	Policy decisions and modes	P0	The policy engine maps detector results and cumulative state to ALLOW, WARN, SANITIZE, BLOCK, or ESCALATE under observe, balanced, and strict modes.
FR-9	Legitimate credential use	P0	Real credentials are resolved by a local credential broker or tool runtime path, not copied into model-visible context as raw secrets.
FR-10	Audit artifacts	P0	Each evaluated turn produces structured JSON or Braintrust trace data containing trace ID, detector results, policy decision, and final action.
FR-11	Demo dashboard	P1	The dashboard shows recent decisions, fired detectors, risk scores, latency, policy mode, and scenario outcome.
FR-12	Evaluation harness	P1	The harness runs benign flows plus encoded leak, multi-turn drip, tool-call exfiltration, honeypot exposure, and benign secret-handle cases with repeatable outputs.
FR-13	Capability-adaptive operation	P1	Cloud/API mode runs without activation access; open-weight introspection is documented as a stronger extension or stretch goal.
FR-14	Optional ML risk probe	P2	A bounded PyTorch detector may score normalized events, but deterministic detectors remain authoritative for high-confidence blocking.

3.7 Non-Functional Requirements

Category	Requirement
Latency	Detector and policy overhead should target under 50ms per gateway turn for the capstone demo; simple detector scoring should remain closer to the sub-10ms target where feasible.
Local operation	The system must run on commodity hardware.
Explainability	Every warning, sanitization, block, or escalation must include structured evidence and a human-readable reason.
Claim discipline	The system must distinguish demo-grade defense from production-grade guarantees.
Modularity	Detectors must be replaceable without changing the proxy entry point.
Testability	Core detectors and policy logic must be unit-testable without a live LLM provider.

Demo reliability	The demo must have a deterministic scripted fallback if external provider access fails.
Trace fallback	If Braintrust is unavailable, the system must write local JSONL traces and continue running.
Log hygiene	Raw content and secrets must be redacted from logs unless the run is explicitly local test mode.
Detector authority	Deterministic detectors and policy evidence must remain authoritative over optional LLM judges or ML probes.

3.8 Success Metrics

Metric	Demo Target	Notes
Encoded single-turn attack detection	Detect most scripted cases	Include Base64, hex, fragmentation, and paraphrase-style attacks without claiming benchmark generality.
Multi-turn drip detection	Trigger a cumulative warning or block before the final scripted leak completes	Demonstrates temporal accounting.
Tool-call exfiltration detection	Block or escalate suspicious arguments in supported tools	Main capstone differentiator.
Benign false blocks	Keep false blocks rare in scripted benign cases	Track warnings separately from blocks.
Explainability	100% of non-allow decisions include evidence	Required for dashboard and audit logs.
Demo readiness	One documented command or script runs the end-to-end demo	Include fallback scripted path.
Experiment evidence	Baseline, observe-mode, and protected-mode runs produce comparable artifacts	Prefer Braintrust links; use local Markdown/JSON summaries as fallback.

4 System Architecture

4.1 SDK and Gateway Architecture

The MVP should be SDK-first. The Python SDK owns the security boundary and exposes guard methods that can be embedded directly in an agent application. The FastAPI gateway is a wrapper around the same SDK for demos, integration tests, and applications that prefer to call Aegis as a local service. This keeps the security logic independent from a single transport path while still producing a usable platform surface.

In the gateway path, the client application calls Aegis before calling an LLM provider or trusted tool. Aegis normalizes the request, tags provenance, resolves secret handles through the credential broker, runs detector stages,

applies policy, forwards only allowed or sanitized data, scans responses, and records trace artifacts. The full architecture diagram is included in Appendix A.

4.2 Guard Surface

The SDK should expose three primary guard points:

Guard	Purpose
<code>guard_request(messages, tools, session_id, metadata)</code>	Scans prompt, retrieved context, and declared tools before a model call.
<code>guard_tool_call(tool_name, arguments, session_id, metadata)</code>	Scans structured tool-call arguments before trusted tool execution.
<code>guard_response(output, session_id, metadata)</code>	Scans model output before it is returned to the client or user.

Each guard returns an `AegisDecision`. The gateway and dashboard should call these same SDK functions rather than duplicate security logic.

4.3 Runtime Contracts

Every guard should create an `AegisEvent` with the following fields:

Field	Meaning
<code>event_id</code>	Stable event identifier for traces and artifacts.
<code>session_id</code>	Conversation or workflow identifier used for cumulative leakage accounting.
<code>phase</code>	One of <code>request</code> , <code>tool_call</code> , or <code>response</code> .
<code>trusted_boundary</code>	One of <code>trusted</code> , <code>untrusted</code> , or <code>mixed</code> .
<code>input_summary</code>	Redacted summary suitable for logs and dashboard display.
<code>raw_content_ref</code>	Optional local reference to raw content in local test mode.
<code>tool_name</code> and <code>tool_arguments</code>	Structured tool-call data when the event phase is <code>tool_call</code> .
<code>secret_handles_seen</code>	Opaque secret handles referenced by the event.
<code>detector_evidence</code>	Detector outputs attached to the event.
<code>policy_decision</code>	Final policy decision and reason summary.
<code>metadata</code>	Provider, scenario, and trace metadata.

The decision contract should include action, risk score, reasons, detector hits, optional sanitized payload, and trace ID. Raw content should be redacted before logs unless the run is explicitly local test mode.

4.4 Implementation Decisions

1. **Language and framework:** Python maximizes velocity, testability, and ML integration within the capstone timeline; FastAPI wraps the SDK as the gateway/API surface.
2. **Entry point:** The SDK is the source of truth for security decisions. The gateway, dashboard, and demo agent call the SDK rather than reimplementing guard logic.
3. **Provider abstraction:** Model/provider calls are hidden behind a small provider interface. The MVP needs one live or mock adapter plus an abstraction clean enough to add a second provider without changing policy logic.
4. **Pipeline:** A Headroom-inspired Inspect–Score–Enforce structure keeps interception, scoring, and policy distinct.
5. **Detector contract:** All detectors return a common structured result.
6. **Policy:** A small YAML policy is loaded at startup and supports observe, balanced, and strict modes.
7. **Credential broker:** The MVP uses a local fake secret store backed by environment variables or a test JSON file. Model-visible context uses opaque handles such as `secret://github/token`.
8. **Canaries:** The MVP supports deterministic registration and matching for a small set of credential families.
9. **Leakage ledger:** The NIMBUS-inspired ledger is a cumulative risk signal, not a formal proof.
10. **Braintrust and fallback traces:** Braintrust should store eval datasets, traces, experiments, scorers, and report links when configured; local JSONL artifacts are required as a fallback.
11. **Optional ML probe:** A small PyTorch risk probe can be added as one detector signal if time allows, but deterministic detectors remain authoritative.
12. **CIFT positioning:** Full activation probing is stretch work; the MVP ships cloud-compatible provenance and behavioral signals.
13. **Dashboard:** Streamlit is the fastest path to useful demo observability.
14. **Evaluation:** Replayable scenarios are promoted into regression cases and comparable baseline/protected experiments.

5 Project Plan

5.1 Team Ownership

Owner	Primary Responsibilities	Secondary Responsibilities
P1: Runtime and SDK lead	Python SDK; guard methods; provider abstraction; gateway wrapper; normalized event and decision contracts.	Policy integration and end-to-end demo wiring.
P2: Defense and detection lead	Secret pattern scanner; encoding scanner; honey-token detector; tool-call argument scanner; leakage ledger; optional ML risk probe.	Detector tests and calibration.

P3: Evaluation and observability lead	Evaluation harness; Braintrust datasets, scorers, traces, and experiments; local JSONL fallback; quantitative demo report.	LLM judge prompt for ambiguous cases, with deterministic scorers authoritative.
P4: Platform, demo, and reporting lead	Dashboard/API views; policy mode display; session/eval result views; README; report export; presentation flow.	Optional environment-plugin prototype or documented plugin path.

5.2 Milestones

Milestone	Target	Definition of Done
SDK guard surface	Day 2	The SDK exposes request, tool-call, and response guards that return structured decisions without running the gateway.
Provider gateway	Day 3	Gateway receives a request, calls the SDK, logs normalized data, forwards or mocks upstream, and returns a response.
Detector pipeline	Day 4	At least two detectors run through a shared interface and produce policy decisions.
Tool-call defense	Day 5	Supported tool-call exfiltration attempts are blocked or escalated with evidence.
Canary and leakage accounting	Day 7	Canary hits and multi-turn budget thresholds appear in audit logs and dashboard.
Evaluation harness	Day 8	Benign and attack scenarios run from repeatable files and produce summary metrics, Braintrust traces, or local fallback artifacts.
Integrated demo	Day 10	Baseline and protected flows run end-to-end with dashboard evidence.
Final polish	Day 11	Demo script, fallback path, metrics table, final narrative, README, and report export are ready.

5.3 Day-by-Day Execution

Day	Deliverables	Acceptance Criteria
1	Repository skeleton; SDK package; shared event and decision models; initial benign and attack scenarios.	Team can import the SDK locally; a mock request produces a normalized event artifact.
2	SDK guard flow for request, tool-call, and response phases; local JSONL tracing; first provider/mock adapter.	Guard methods return structured decisions and redacted traces.
3	Gateway wrapper; detector interface; policy schema; YAML loader; static canary and credential-shape detectors.	Gateway path calls the SDK; detectors emit evidence; policy maps results to actions.

4	Tool schemas for <code>send_email</code> , <code>http_request</code> , and <code>query_database</code> ; argument scanner; provenance checks; unit tests.	Scripted tool-call leaks are blocked or escalated; benign calls are allowed.
5	Local fake secret store; credential broker; secret handles; canary registry; format-matched honeytokens.	Raw real credentials are unnecessary in model-visible context; canary hits trigger non-allow actions.
6	Per-session leakage ledger; observe/balanced/strict policy modes; multi-turn drip scenario.	Cumulative attack triggers before final leak; benign multi-turn scenario stays below block threshold.
7	YAML scenario format; evaluation runner; deterministic scorers; Braintrust integration with local fallback.	Harness produces reproducible metrics with either Braintrust links or local artifacts.
8	Dashboard/API views for policy mode, recent sessions, detector evidence, metrics, and scenario detail.	Dashboard updates from fresh eval output and explains non-allow decisions.
9	Baseline path; protected path; side-by-side summary; encoded, drip, tool-call, honeytoken, and benign scenarios.	Baseline attempts leakage; protected path intervenes with evidence in under 10 minutes.
10	Optional ML risk probe if stable; threshold tuning; explicit startup/config errors; fallback demo mode.	Demo works without network provider, Braintrust, or ML model; false blocks remain low in scripted benign cases.
11	Final metrics table; architecture appendix; README; report export; final demo script; limitation slide.	Team can rehearse the full demo twice without manual debugging.

6 Detector and Policy Design

6.1 Detector Contract

Every detector should return a common logical shape:

Field	Meaning
<code>detector_name</code>	Stable name, such as <code>tool_call_argument_scanner</code> .
<code>score</code>	Risk score from 0.0 to 1.0.
<code>confidence</code>	Detector confidence from 0.0 to 1.0.
<code>recommended_action</code>	One of <code>ALLOW</code> , <code>WARN</code> , <code>SANITIZE</code> , <code>BLOCK</code> , or <code>ESCALATE</code> .
<code>evidence</code>	Structured detector-specific proof.
<code>latency_ms</code>	Detector runtime.

For tool-call scanning, evidence should include tool name, argument name, argument value preview, risk reason, whether the value appeared in trusted context, whether it matched a credential pattern, and any matched canary

identifier. For leakage accounting, evidence should include turn score, cumulative score, thresholds, and session ID. For canary detection, evidence should include canary ID, service, location, and session ID.

6.2 MVP Detector Stages

Detector Stage	Scope
Secret pattern scanner	Detect API keys, tokens, private key blocks, connection strings, and service-specific credential shapes.
Encoding scanner	Decode common encodings such as Base64, hexadecimal, URL encoding, and simple split-token reconstructions before applying secret and canary scans.
Honeytoken detector	Match exact and normalized canaries in model outputs and tool-call arguments.
Tool-call argument scanner	Inspect structured JSON arguments before dispatch to trusted tools.
Nimbus-lite ledger	Maintain a per-session cumulative leakage score for low-rate multi-turn leakage.
Optional ML risk probe	Score normalized events using features such as detector hits, entropy, decoded payload indicators, suspicious instruction terms, secret-handle references, and cumulative leakage score.
CIFT adapter interface	Define a non-blocking stretch interface for white-box activation probes without requiring white-box model access in the MVP.

The optional ML probe should be a policy input, not a policy owner. If the model artifact is missing, slow, or disabled, the system must continue with deterministic detectors and record the degraded mode in traces.

6.3 Initial YAML Policy Scope

The first policy file should support four rule types:

1. Detector score threshold.
2. Tool argument condition.
3. Canary hit.
4. Leakage budget threshold.

The MVP should avoid nested logical combinations. Rules can be evaluated independently, and the policy engine can choose the most severe resulting action.

6.4 Policy Modes

Mode	Behavior
------	----------

observe	Never blocks; records detector evidence, risk scores, and recommended actions for baseline comparison and tuning.
balanced	Blocks or sanitizes high-confidence leaks, honeypot exposure, tool-call exfiltration, and budget exhaustion; warns on ambiguous cases.
strict	Blocks most suspicious credential, encoding, and provenance anomalies; useful for demonstrating conservative security posture and false-positive tradeoffs.

6.5 Credential Broker Scope

Real credentials should not be inserted into model-visible context. The MVP credential broker should resolve opaque handles such as `secret://github/token` inside trusted tool execution. The local fake secret store can read from environment variables or a test JSON file. Production secret-manager integration, rotation, incident response, and cloud IAM are out of scope for June 29.

If a raw real secret enters prompt or response logs, the broker or guard layer should redact the value, emit a critical trace event, and force a non-allow policy decision unless the run is explicitly marked as local test mode.

7 Evaluation Plan

7.1 Scenario Categories

Category	Scenario Examples
Benign normal usage	Normal email, safe HTTP request, ordinary database query.
Encoded single-turn leakage	Base64, hex, fragmentation, paraphrase, or transformed credential disclosure.
Multi-turn dripping	Small fragments leaked across turns, each below a per-turn threshold but above cumulative budget.
Tool-call argument exfiltration	Credential-shaped value placed in email body, HTTP query parameter, or database query string.
Canary touches	Registered honeypot appears in output or tool argument.
Benign secret-handle usage	Valid handle-based credential use where the model never sees the raw secret.
False-positive benign text	Developer documentation or examples that resemble credentials but should not be blocked in balanced mode.

7.2 Metrics

1. Detection rate by scenario category.
2. False block count on benign scenarios.
3. Warning count on benign scenarios.
4. Average gateway latency.

5. Detector hit distribution.
6. Number of scenarios with complete structured evidence.

7.3 Braintrust and Artifact Plan

Braintrust should be the hosted evidence loop when credentials and network access are available. Local JSONL and Markdown artifacts are required so the demo and report can still run without hosted observability.

Artifact	Purpose
Datasets	Separate red-team and benign cases for repeatable baseline and protected experiments.
Traces	Record request guard, detector stages, policy decision, tool-call guard, response guard, and eval case result.
Scorers	Track leak detection, correct block, false positive, latency overhead, policy decision quality, and optional ML-probe contribution.
Experiments	Compare baseline vulnerable agent, Aegis observe mode, and Aegis balanced mode.
Reports	Export Braintrust links when available and local Markdown/JSON summaries in all modes.

LLM judges may help score ambiguous cases or summarize reports, but deterministic scorers remain the source of truth for blocking and headline metrics.

7.4 Demo Metrics Table

Scenario	Baseline Result	Aegis Result	Evidence
Encoded leak	Secret exposed or transformed.	Warning, blocking, or sanitization.	Detector score and reason.
Multi-turn drip	Fragments accumulate.	Budget threshold trips.	Cumulative score.
Tool-call exfiltration	Secret sent through tool arguments.	Block before dispatch.	Tool, argument, reason.
Honeytoken exposure	Canary is revealed.	Block or escalate.	Canary ID and location.
Benign handle usage	Secret handle is used correctly.	Allow.	Handle reference, no raw secret.

8 Testing Strategy

8.1 Unit Tests

The core security behavior should be unit-tested without a live LLM provider:

1. Secret pattern scanner detects direct credentials and avoids common benign examples.
2. Encoding scanner decodes Base64, hexadecimal, URL-encoded, and split-token variants before scanning.
3. Honeypot detector matches exact and normalized canaries.
4. Tool-call argument scanner catches suspicious email, HTTP, and database arguments.
5. Nimbus-lite budget accumulates low-rate leakage across turns.
6. Policy engine maps detector evidence to observe, balanced, and strict decisions.

8.2 Integration and Eval Tests

Integration tests should prove that a vulnerable baseline leaks a fake secret in at least one scripted attack while the protected path blocks direct secret requests, encoded secret requests, tool-call argument exfiltration, and honeypot exposure. Protected flows must also allow benign secret-handle usage. Eval tests should verify that each scenario records input, output, policy decision, detector scores, latency, trace metadata, and local fallback artifacts.

9 Failure Modes and Guardrails

Failure Mode	Impact	Guardrail
Braintrust API key missing	No hosted traces	Write local JSONL traces and continue.
LLM judge returns malformed output	Bad eval score	Deterministic scorers remain authoritative.
ML probe missing or slow	Runtime failure or latency	Disable the ML detector stage and fall back to deterministic detectors.
ML probe overfits synthetic cases	Misleading confidence	Track false positives and ML contribution separately.
Encoded leak bypasses scanner	Credential exposure	Decode common encodings before scanning and add bypasses to regression cases.
Tool-call args bypass response guard	Credential exposure	Guard tool calls before dispatch, not only after response generation.
Raw real secret enters prompt	High-risk leak	Broker asserts, redacts, emits a critical trace, and forces non-allow policy.
Multi-turn drip evades single-turn checks	Gradual leak	Maintain a session-level Nimbus-lite leakage budget.
Platform scope grows into full SaaS	Missed deliverable	Keep the platform local/deployable: gateway API, policy config, sessions, eval results, and reports.

10 Risk Register

Risk	Impact	Mitigation
Provider/gateway compatibility takes longer than expected.	Core demo may slip.	Start with one provider-compatible chat and tool-call shape; use mocks for nonessential providers.
SDK/gateway split is unclear.	Security logic may diverge across entry points.	Make the SDK the only source of security decisions; the gateway calls SDK guards.
Tool-call scanner becomes too broad.	Detector quality drops.	Scope to three high-risk tool schemas and exact structured fields.
False positives make the demo look brittle.	Users distrust the system.	Include benign credential-use cases and distinguish warnings from blocks.
CIFT implementation exceeds timeline.	ML component underwhelms.	Position activation probing as stretch; ship cloud-compatible provenance and behavioral signals.
Braintrust integration slips.	Evidence loop weakens.	Require local JSONL and Markdown summaries as fallback artifacts.
NIMBUS-inspired score is over-claimed.	Technical credibility suffers.	Call it cumulative leakage scoring, not a formal leakage bound.
Dashboard consumes too much time.	Core defenses suffer.	Keep dashboard to recent decisions, metrics, and scenario selector.
Fourth teammate is unavailable.	Team capacity shrinks.	Plan for three owners and reserve optional work for P4.

11 Demo Narrative

The 10-minute presentation should proceed in eight stages:

1. State the problem: agents mix trusted credentials with untrusted content.
2. Show why text-only defenses fail against encoded and multi-turn leaks.
3. Introduce Aegis as an SDK-backed runtime layer with Inspect–Score–Enforce.
4. Run the baseline agent through encoded, multi-turn, tool-call, and honeypot attacks.
5. Run the Aegis-protected agent through the same attacks.
6. Show dashboard, trace, and eval evidence for each intervention.
7. Explain the research lineage from AIS to Aegis, emphasizing the tool-call argument extension.
8. Close with limitations and production requirements.

12 Limitations

Aegis should be presented with disciplined claims. The capstone system is not production-ready. Cloud/API model support cannot provide true CIFT-style activation monitoring. The leakage ledger is a cumulative signal, not a

formal security proof. The tool-call scanner is scoped to supported schemas. Braintrust integration is an evidence mechanism, not a defense mechanism. The optional ML risk probe is not required for blocking and should not be overclaimed. A determined adaptive attacker may find paths around MVP rules. A production deployment would need stronger secret-manager integration, persistence, access control, broader schema coverage, and independent red-team validation.

13 Conclusion

Aegis converts a research insight into a practical capstone system. AIS suggests that credential exfiltration cannot be handled by text-only output filtering; it requires attention to pre-output access, canaries, and cumulative leakage. Aegis accepts the limits of a two-week build and focuses on the most compelling deployable gap: structured tool-call arguments. The expected outcome is a working SDK-backed gateway, evaluation harness, trace/report loop, and live dashboard that visibly outperform a baseline agent on encoded leakage, multi-turn leakage, honeypot exposure, and tool-call argument exfiltration while preserving benign workflows.

A Architecture Diagram

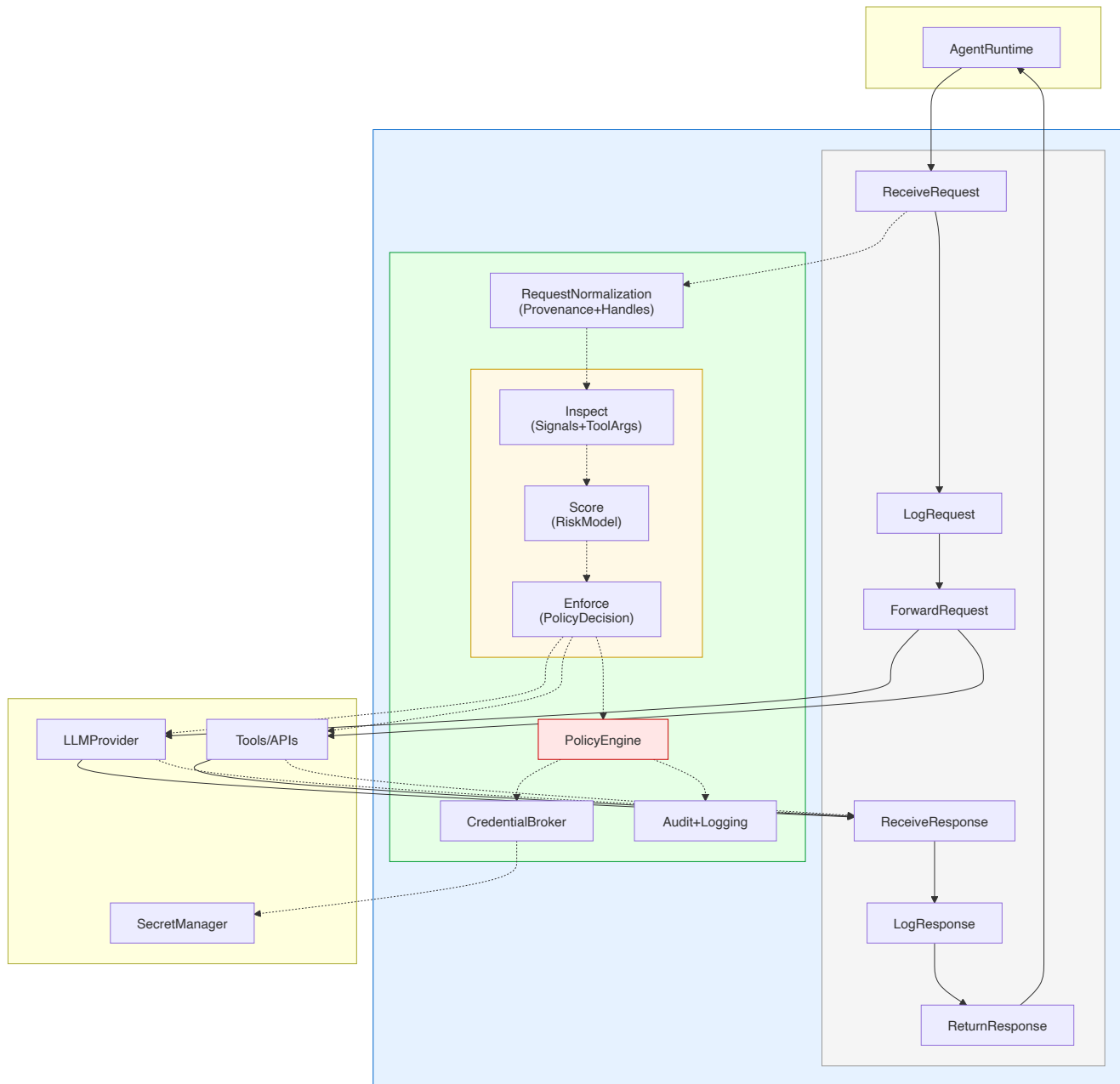


Figure 1: Aegis gateway architecture. The solid path is the observation-only proxy required first; the dashed path shows where normalization, the defense pipeline, policy, credential brokering, and audit logging enter the same gateway.

References

- [1] Gauntlet AI. *Capstone Kickoff Brief*. 2026.
- [2] Chauhan, K., and Revankar, P. *Caught in the Act(ivation): Toward Pre-Output and Multi-Turn Detection of Credential Exfiltration by LLM Agents*. arXiv:2606.04141, 2026.
- [3] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. *Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. arXiv:2302.12173, 2023.
- [4] DeBenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., and Tramer, F. *AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents*. arXiv:2406.13352, 2024.
- [5] Zhan, Q., Liang, Z., Ying, Z., and Kang, D. *InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents*. arXiv:2403.02691, 2024.